Data Ingestion Architecture

Warren Buffett Intelligence Hub — RAG Chatbot

Table of Contents

1. Overview
2. Repository Layout and Active Files
3. Data Sources
4. Configuration Layer
5. PDF Loading and Text Extraction
6. Text Chunking Strategy
7. CSV Loading, Filtering, and Deduplication
8. Document Formatting and Metadata Tagging
9. Document Merging
10. Embedding Model
11. FAISS Vector Index Construction
12. BM25 Keyword Index Construction
13. Output Artifacts
14. Smoke Test
15. End-to-End Data Flow Summary

---

1. Overview

The data ingestion pipeline is the offline, one-time preprocessing step that transforms raw source documents into a searchable knowledge base for the Warren Buffett RAG (Retrieval-Augmented Generation) chatbot. It is implemented entirely in `ingest.py` at the project root and is executed once before the application server is started.

The pipeline performs the following high-level operations:

- Loads and chunks a 1,010-page PDF of Berkshire Hathaway shareholder letters
- Loads, deduplicates, and formats approximately 6,000 curated Q&A pairs from 7 CSV files
- Embeds all documents into dense vector representations using a sentence transformer model
- Stores the resulting embeddings in a FAISS (Facebook AI Similarity Search) vector index for semantic similarity search
- Tokenizes all documents and builds a BM25 keyword index over the same corpus for exact keyword matching
- Both indexes are stored in `vectorstore/` and loaded together at query time to support hybrid retrieval

The vectorstore produced by this pipeline lives at `vectorstore/` in the project root and is gitignored — it must be regenerated locally by running:

```
python ingest.py
```

---

2. Repository Layout and Active Files

The project root contains all active Python modules. A legacy `src/` directory exists in the repository but is no longer used — it contains an older version of `ingest.py`, `main.py`, and `rag_chat.py` that were superseded during development.

The active files relevant to data ingestion are:

File	Role
`ingest.py`	Main ingestion script — orchestrates the entire pipeline
`config.py`	Centralizes all paths, model names, and hyperparameters
`retriever.py`	Contains `build_bm25_index()` which is called from `ingest.py`
`data/`	Directory containing the PDF and all CSV files
`vectorstore/`	Output directory — stores `index.faiss`, `index.pkl`, and `bm25.pkl`

---

3. Data Sources

3.1 PDF — Berkshire Hathaway Shareholder Letters

File: `data/All-Berkshire-Hathaway-Letters.pdf` **Size:** 1,010 pages

This PDF is a comprehensive compilation of Warren Buffett's annual shareholder letters to Berkshire Hathaway investors. These letters span several decades and cover Buffett's investment philosophy, business principles, capital allocation decisions, commentary on market conditions, and reflections on specific portfolio companies. The letters are written in Buffett's characteristic plain-English style and are rich with domain knowledge about value investing, accounting, insurance economics, and corporate governance.

The PDF is loaded in its raw form and is subjected to chunking before embedding, since the letters are far too long to embed as single documents.

3.2 CSV Files — Curated Q&A Dataset

Total pairs after deduplication: ~6,000

Seven CSV files are ingested, each covering a specific thematic domain of Warren Buffett's knowledge and life:

File	Theme
`Warren Buffett Data - Personal life.csv`	Biography, family, habits, values
`Warren Buffett Data - Adaptability.csv`	Evolving investment views, learning from mistakes
`Warren Buffett Data - Psychology.csv`	Mental models, cognitive biases, decision-making

File	Theme
`Warren Buffett Data - Risk Management.csv`	Hedging, margin of safety, concentration vs. diversification
`Warren Buffett Data - Strategy Development.csv`	Business moats, competitive advantage, acquisition criteria
`Warren Buffett Data - Timing.csv`	Market timing, patience, long-term holding
`Dataset_Warren_Buffet_Clean.csv`	General cleaned dataset across multiple categories

All CSVs share a common schema:

```
Questions, Answers, Label
```

The `Label` column provides a thematic category tag (e.g., `"Risk Management"`, `"Strategy & Moats"`) that is later used to generate source metadata on retrieved documents.

---

4. Configuration Layer

All ingestion parameters are centralized in `config.py`. This design ensures there are no hardcoded values in `ingest.py` and that changing any parameter requires a single edit in one place.

```
PDF_PATH = os.path.join(DATA_DIR, "All-Berkshire-Hathaway-Letters.pdf")

CSV_FILES = [
    "Warren Buffett Data - Personal life.csv",
    "Warren Buffett Data - Adaptability.csv",
    "Warren Buffett Data - Psychology.csv",
    "Warren Buffett Data - Risk Management.csv",
    "Warren Buffett Data - Strategy Development.csv",
    "Warren Buffett Data - Timing.csv",
    "Dataset_Warren_Buffet_Clean.csv",
]

VECTORSTORE_DIR  = "vectorstore"
CHUNK_SIZE       = 1000       # characters per chunk
CHUNK_OVERLAP    = 200        # overlap between consecutive chunks
EMBEDDING_MODEL  = "sentence-transformers/paraphrase-MiniLM-L6-v2"
TOP_K            = 5          # chunks retrieved at query time
```

The `VECTORSTORE_DIR` resolves to `vectorstore/` at the project root, which is the only active vector store directory. The `vector_store/` directory (with underscore) is not used by the application.

---

5. PDF Loading and Text Extraction

PDF loading is handled by LangChain's `PyPDFLoader`:

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader(PDF_PATH)
pages = loader.load()
```

`PyPDFLoader` internally uses the `pypdf` library to parse each page of the PDF. It returns a list of `Document` objects, one per page, where:

- `page_content` holds the extracted text of that page
- `metadata` holds the `source` (file path) and `page` (page number, zero-indexed)

For a 1,010-page PDF, this step produces 1,010 `Document` objects before chunking.

Important characteristic: PDF text extraction is imperfect. Shareholder letters written in earlier decades may have been scanned and OCR'd, which can introduce noise in the extracted text (hyphenation artifacts, incorrect character substitutions). The pipeline does not apply post-extraction cleaning, so some noise may persist in chunks derived from older letters.

---

6. Text Chunking Strategy

Because individual pages can be very long and the embedding model operates most effectively on shorter, focused passages, pages are split into overlapping chunks using `RecursiveCharacterTextSplitter`:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,      # 1000 characters
    chunk_overlap=CHUNK_OVERLAP # 200 characters
)
chunks = splitter.split_documents(pages)
```

6.1 How RecursiveCharacterTextSplitter Works

`RecursiveCharacterTextSplitter` splits text using a priority-ordered list of separators: `["\n\n", "\n", " ", ""]`. It attempts to split at paragraph boundaries first (`\n\n`), falling back to line breaks (`\n`), then word boundaries (`" "`), and finally character-level splits as a last resort.

This recursive approach preserves semantic coherence — a chunk is far less likely to cut mid-sentence than a naive fixed-length splitter would.

6.2 Chunk Size and Overlap Parameters

- **Chunk size: 1,000 characters.** This produces passages that are long enough to contain meaningful context (a full paragraph or two) but short enough for the embedding model to represent accurately. At approximately 4 characters per token, this corresponds to roughly 250 tokens per chunk.

- **Overlap: 200 characters.** Each consecutive pair of chunks shares 200 characters of text. This sliding window approach ensures that information spanning a chunk boundary is not lost — a sentence that falls at the end of one chunk will also appear at the beginning of the next. This is critical for retrieval quality, as a query about a topic discussed across a paragraph boundary will still find a relevant chunk.

6.3 Metadata Injection

After splitting, every PDF-derived chunk receives a `source` tag:

```
for chunk in chunks:
    chunk.metadata["source"] = "shareholder_letter"
```

This tag is used downstream in `rag_chain.py` to label retrieved documents with their origin when returning source citations to the user.

The 1,010-page PDF typically produces in the range of 4,000–6,000 chunks depending on page length variation.

---

7. CSV Loading, Filtering, and Deduplication

CSV ingestion is handled by the `load_csvs()` function, which iterates over all seven files defined in `config.CSV_FILES`.

7.1 Column Handling

The function is designed to tolerate two different column naming conventions across the CSV files:

```
question = (row.get("Questions") or row.get("question", "")).strip()
answer   = (row.get("Answers")   or row.get("answer",   "")).strip()
label    = (row.get("Label")     or row.get("refined_category") or "").strip()
```

This dual-key lookup means the pipeline is robust to CSV files that use either `Questions`/`Answers` (the primary schema) or `question`/`answer` (an alternative schema present in some files). Empty rows are skipped.

7.2 Filtering: Expanded Perspective Rows

A specific class of rows is filtered out:

```
if "(expanded perspective)" in question.lower():
    continue
```

Some CSV files contain augmented versions of questions with `"(expanded perspective)"` appended to the question text. These are synthetic expansions generated to add variety to the dataset but introduce noise,

since the expanded question text does not match how a real user would phrase a query. Filtering these rows improves retrieval precision by keeping only natural-language questions.

7.3 Cross-File Deduplication

A `seen` set tracks `(question, answer)` tuples across all CSV files:

```
seen = set()
key = (question, answer)
if key in seen:
    continue
seen.add(key)
```

Because the seven CSV files have overlapping content (particularly `Dataset_Warren_Buffet_Clean.csv` which aggregates data from multiple sources), exact duplicates would otherwise be embedded twice, leading to redundant entries in the FAISS index and inflating retrieval scores for duplicated content. The deduplication step guarantees that each unique Q&A pair is embedded exactly once across the entire corpus.

---

8. Document Formatting and Metadata Tagging

Each accepted Q&A pair is formatted into a single string and wrapped in a LangChain `Document` object:

```
content = f"Question: {question}\nAnswer: {answer}"

docs.append(Document(
    page_content=content,
    metadata={
        "source": source_tag,
        "question": question
    }
))
```

8.1 The Question/Answer Format

By concatenating the question and answer with explicit `Question:` and `Answer:` prefixes, the embedding model is given the full semantic context of both the query and its response. At retrieval time, when a user asks a question, the cosine similarity between the user's embedded query and the embedded `"Question: ...\nAnswer: ..."` string tends to be high when the stored question is semantically similar to the user's question.

This is more effective than embedding only the answer text, because the question string anchors the embedding to the intent of the Q&A pair, making it easier for FAISS to surface the right document.

8.2 Source Tag Generation

The `Label` column value is transformed into a normalized source tag:

```
source_tag = f"qa_{label.lower().replace(' ', '_').replace('&', 'and')}" if label
else "qa"
```

For example:

- `"Risk Management"` → `"qa_risk_management"`
- `"Strategy & Moats"` → `"qa_strategy_and_moats"`
- `"Personal life"` → `"qa_personal_life"`

This tag is stored in the document's metadata and surfaced to the user when the chatbot cites its sources, helping users understand which thematic area of Buffett's knowledge a retrieved passage comes from.

The `question` field is also stored in metadata to enable question-level traceability in retrieved results.

---

9. Document Merging

After both loaders complete, all documents are merged into a single list:

```
all_docs = pdf_docs + csv_docs
```

PDF chunks come first, followed by CSV documents. The combined corpus is then passed as a single batch to the embedding model and FAISS index builder. At this stage, `all_docs` contains:

- All text chunks from the 1,010-page shareholder letter PDF
- All deduplicated, filtered, and formatted Q&A pairs from the 7 CSV files (~6,000 pairs)

The total number of documents in the index is typically in the range of 10,000–12,000, depending on how many PDF chunks are produced.

---

10. Embedding Model

The embedding model converts each document's text into a dense numerical vector that captures its semantic meaning.

Model: `sentence-transformers/paraphrase-MiniLM-L6-v2`

Loaded via LangChain's `HuggingFaceEmbeddings` wrapper:

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
```

10.1 Model Architecture

`paraphrase-MiniLM-L6-v2` is a fine-tuned variant of the MiniLM architecture trained specifically for semantic similarity and paraphrase detection tasks. It is a distilled transformer with 6 attention layers (hence `L6`), producing 384-dimensional dense embeddings.

Key characteristics:

Property	Value
Architecture	MiniLM (distilled BERT)
Layers	6
Output dimensions	384
Max input tokens	512
Model size	~80MB
Training objective	Paraphrase detection / semantic similarity

10.2 Why This Model

- **Efficiency:** At only ~80MB, it runs comfortably on CPU without a GPU, making the ingestion step feasible on any development machine.
- **Paraphrase sensitivity:** Because it was trained on paraphrase pairs, it is effective at recognizing when two differently-worded texts express the same idea — crucial for matching user questions to Q&A pairs that may phrase the same concept differently.
- **Speed:** Embedding 10,000+ documents on CPU takes a few minutes, making the one-time ingestion cost acceptable.

10.3 Embedding Process

LangChain's `FAISS.from_documents()` internally calls the embedding model on each document's `page_content` and builds the index:

```
vectorstore = FAISS.from_documents(all_docs, embeddings)
```

Each document's text is tokenized, passed through the transformer, and the final hidden states are mean-pooled to produce a single 384-dimensional vector per document. These vectors are normalized and stored in the FAISS index.

---

11. FAISS Vector Index Construction

FAISS (Facebook AI Similarity Search) is a library optimized for efficient similarity search over large collections of dense vectors.

11.1 Index Type

LangChain's `FAISS.from_documents()` builds a flat (brute-force) index by default — specifically an `IndexFlatL2` index, which stores all vectors in memory and performs exact nearest-neighbor search using L2 (Euclidean) distance.

For a corpus of 10,000–12,000 documents with 384-dimensional vectors, the index is small enough that an exact search index is practical. Approximate nearest neighbor (ANN) indexes (like HNSW or IVF) are not needed at this scale.

11.2 Similarity Metric

FAISS uses L2 distance for search, but LangChain normalizes embeddings before storage so that L2 distance is equivalent to cosine similarity. This means retrieval ranks documents by how directionally similar their embedding vectors are to the query embedding — a standard measure of semantic similarity.

11.3 Persistence

After construction, the index is saved to disk:

```
vectorstore.save_local(VECTORSTORE_DIR)
```

This writes two files to `vectorstore/`:

- **`index.faiss`** — The binary FAISS index file containing all embedding vectors
- **`index.pkl`** — A Python pickle file containing the mapping from FAISS vector IDs back to the original `Document` objects (including `page_content` and `metadata`)

At query time, `retriever.py` loads both files to reconstruct the vectorstore and perform similarity searches.

---

12. BM25 Keyword Index Construction

In addition to the FAISS semantic index, `ingest.py` also builds a BM25 keyword search index over the same document corpus. This is done by calling `build_bm25_index()` from `retriever.py`:

```
from retriever import build_bm25_index
build_bm25_index(all_docs)
```

12.1 What is BM25

BM25 (Best Match 25) is a classical information retrieval algorithm based on term frequency and inverse document frequency (TF-IDF), with additional normalization for document length. Unlike the FAISS index which operates on continuous vector spaces, BM25 is a sparse, keyword-based retrieval method.

BM25 excels at exact keyword matching — if a user mentions a specific company name, a year, or a technical term that appears verbatim in a document, BM25 will rank that document highly even if the semantic embedding is not especially close.

12.2 Tokenization

Documents are tokenized using a simple whitespace and lowercase tokenizer defined in `retriever.py`:

```
def tokenize(text):
    return re.findall(r'\w+', text.lower())
```

This produces a list of lowercase word tokens, stripping punctuation. The tokenizer is intentionally simple — no stemming or stopword removal is applied, which keeps the index deterministic and avoids over-aggressive normalization.

12.3 Index Construction and Persistence

The `rank_bm25` library's `BM25Okapi` implementation is used:

```
corpus = [tokenize(doc.page_content) for doc in documents]
bm25 = BM25Okapi(corpus)
```

`BM25Okapi` is the Okapi BM25 variant with tunable parameters `k1` (term saturation) and `b` (length normalization) — `rank_bm25` uses standard defaults of `k1=1.5` and `b=0.75`.

The index is serialized to disk alongside the FAISS index:

```
with open(BM25_PATH, "wb") as f:
    pickle.dump({"bm25": bm25, "docs": documents}, f)
```

This writes `vectorstore/bm25.pkl`, which contains both the BM25 index object and the full list of `Document` objects (needed to map BM25 result indices back to document content).

12.4 Role at Query Time

At query time, `retriever.py` combines results from FAISS and BM25 using Reciprocal Rank Fusion (RRF), then reranks the merged candidate set using a cross-encoder model (`cross-encoder/ms-marco-MiniLM-L-6-v2`). This hybrid retrieval approach is more robust than either method alone — FAISS handles paraphrase and semantic matches, BM25 handles exact keyword matches, and the cross-encoder reranker applies a more precise relevance score to the merged candidate set.

---

13. Output Artifacts

After a successful run of `ingest.py`, three files are written to `vectorstore/`:

File	Description
`vectorstore/index.faiss`	Binary FAISS flat index containing all 384-dim embedding vectors

File	Description
`vectorstore/index.pkl`	Pickle mapping FAISS vector IDs to LangChain Document objects
`vectorstore/bm25.pkl`	Pickle containing the BM25Okapi index and document list

All three files are required at runtime. The entire `vectorstore/` directory is gitignored and must be regenerated on each new machine by running `python ingest.py`.

---

14. Smoke Test

At the end of `ingest.py`, a built-in smoke test runs four representative queries against the newly built FAISS index:

```
test_queries = [
    "What is Buffett's opinion on debt?",
    "How did Buffett overcome his fear of public speaking?",
    "What happened with Berkshire's textile operations?",
    "How does Buffett manage risk?",
]
```

For each query, the top 3 results are printed with their source tag and a 120-character preview of the document content. This provides immediate validation that the index is populated and returning sensible results before the server is started.

---

15. End-to-End Data Flow Summary

```
Data Sources
    │
    ├── PDF (1010 pages)
    │     └── PyPDFLoader → 1010 page Documents
    │           └── RecursiveCharacterTextSplitter (1000 chars / 200 overlap)
    │                 └── ~4000–6000 PDF chunks
    │                       └── metadata: source = "shareholder_letter"
    │
    └── 7 CSV files (~6000 Q&A pairs)
          └── csv.DictReader (per file)
                ├── Filter: skip empty rows
                ├── Filter: skip "(expanded perspective)" rows
                ├── Deduplicate: skip (question, answer) seen across files
                └── Format: "Question: ...\nAnswer: ..."
                      └── metadata: source = "qa_<label>", question = <question>

Combined corpus: ~10,000–12,000 Documents
    │
    ├── HuggingFaceEmbeddings (paraphrase-MiniLM-L6-v2)
    │     └── 384-dim dense vector per document
    │
```

```
      ├── FAISS.from_documents()
      │     └── IndexFlatL2 over all 384-dim vectors
      │           ├── vectorstore/index.faiss
      │           └── vectorstore/index.pkl
      │
      └── build_bm25_index() — BM25Okapi over tokenized documents
            └── vectorstore/bm25.pkl
```

At query time, `retriever.py` loads all three artifacts and runs FAISS + BM25 hybrid search with cross-encoder reranking to retrieve the most relevant documents for each user question.

Document prepared for CSYE7380 Final Project presentation — Warren Buffett Intelligence Hub